SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: Database Management Systems
COURSE CODE: CSA05

COURSE OUTCOME COVERED

CO3: *Analyze relational databases using functional dependencies, normalization (1NF to 5NF), and key constraints to identify redundancy and ensure data integrity.* (BL2, BL3)

Activity Tool : Experiments (High)
Assessment Tool Weightage: 40%

QUESTIONS		Total Marks: 50	
Q.No	**Question**	**Bloom's Level**	**Marks**
1	For the relation STUDENT_COURSE(SID, SName, CID, CName, Instructor, Grade), identify all functional dependencies and determine the candidate keys.	BL2 – Understand	**5**
2	Demonstrate the process of converting an unnormalized relation into 1NF with step-by-step justification and example.	BL3 – Apply	**5**
3	Given relation R(A,B,C,D,E) with FDs: A→BC, BC→D, D→E. Analyze and decompose it to 2NF eliminating partial dependencies.	BL3 – Apply	**5**
4	Apply 3NF decomposition on the relation EMPLOYEE(EmpID, EmpName, DeptID, DeptName, ManagerID) with transitive dependencies.	BL3 – Apply	**5**
5	Verify whether R(A,B,C,D) with FDs: AB→C, C→D, D→A is in BCNF. If not, decompose it into BCNF.	BL3 – Apply	**5**
6	Experimentally verify lossless-join decomposition for a given relation using the tabular (Chase) algorithm.	BL3 – Apply	**5**
7	Identify the multi-valued dependencies in TEACHER(TID, Subject, Hobby) and decompose it into 4NF.	BL3 – Apply	**5**

8	Demonstrate how join dependencies lead to redundancy. Decompose a given relation to 5NF (Project-Join Normal Form).	BL3 – Apply	**5**
9	Given a poorly designed database schema for a college, identify all anomalies (insertion, deletion, update) and normalize it to 3NF.	BL2 – Understand	**5**
10	Compute the closure of attribute sets for the given FDs and determine all candidate keys for the relation.	BL3 – Apply	**5**

Code of Conduct:

I Sree B (192524023) *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student: Sree B

1.For the relation STUDENT_COURSE(SID, SName, CID, CName, Instructor, Grade), identify all functional dependencies and determine the candidate keys.

For the relation:
STUDENT_COURSE(SID, SName, CID, CName, Instructor, Grade)
Assume:

- SID = Student ID
- SName = Student Name
- CID = Course ID
- CName = Course Name
- Instructor = Course Instructor
- Grade = Grade obtained by the student

1. Functional Dependencies
The main functional dependencies are:

1. SID → SName
   A student ID uniquely determines the student's name.
2. CID → CName
   A course ID uniquely determines the course name.
3. CID → Instructor
   A course ID uniquely determines its instructor.
4. (SID, CID) → Grade
   A particular student in a particular course determines the grade.

Therefore, we can write the combined FD:
(SID, CID) → SName, CName, Instructor, Grade
2. Candidate Key
Consider {SID, CID}:

- SID → SName
- CID → CName, Instructor
- SID + CID → Grade

Therefore:
(SID, CID)+ = {SID, SName, CID, CName, Instructor, Grade}
So, {SID, CID} determines all attributes.
Neither SID nor CID alone can determine all attributes.
Answer
Functional Dependencies:

- SID → SName
- CID → CName
- CID → Instructor
- (SID, CID) → Grade

Candidate Key:
{SID, CID} is the only candidate key.

2.Demonstrate the process of converting an unnormalized relation into 1NF with step-by-step justification and example.

Converting an Unnormalized Relation (UNF) into 1NF

1NF (First Normal Form) means:

- Each cell contains **only one value**.
- There are **no repeating groups** or multi-valued attributes.
- Each row can be uniquely identified by a key.

Example: Unnormalized Relation

Suppose we have a student table:

SID	SName	Courses	Grades
101	Ravi	DBMS, Java	A, B
102	Priya	Python, AI, DBMS	A, A+, B

Here, **Courses** and **Grades** contain multiple values in a single cell. Therefore, the relation is **not in 1NF**.

Step 1: Identify repeating groups

For student **101**:

- Courses = DBMS, Java
- Grades = A, B

For student **102**:

- Courses = Python, AI, DBMS
- Grades = A, A+, B

These are repeating/multi-valued attributes.

Step 2: Separate the repeating values

Create one row for **each student-course combination**.

Step 3: Form the 1NF Relation

STUDENT_COURSE(SID, SName, Course, Grade)

SID	SName	Course	Grade
101	Ravi	DBMS	A
101	Ravi	Java	B
102	Priya	Python	A
102	Priya	AI	A+
102	Priya	DBMS	B

Now, every cell contains **exactly one atomic value**.

Step 4: Identify the Primary Key

A student can take multiple courses, so SID alone is not unique.

Therefore, the combination:

(SID, Course)

uniquely identifies each record.

Final Result

UNF:

STUDENT(SID, SName, {Course, Grade})

Remove repeating groups

1NF:

STUDENT_COURSE(SID, SName, Course, Grade)

Primary Key: (SID, Course)

Justification

The relation is in **1NF** because:

1. Every attribute contains a **single atomic value**.
2. There are **no repeating groups**.
3. Each record is uniquely identified by **(SID, Course)**.

3. Given relation R(A,B,C,D,E) with FDs: A→BC, BC→D, D→E. Analyze and decompose it to 2NF eliminating partial dependencies.

Given Relation
R(A, B, C, D, E)
Functional Dependencies:

- A → BC
- BC → D
- D → E

We need to decompose it into 2NF by eliminating partial dependencies.
Step 1: Find the Candidate Key
Find the closure of A:
$A^+ = \{A\}$
Using A → BC:
$A^+ = \{A, B, C\}$
Using BC → D:
$A^+ = \{A, B, C, D\}$
Using D → E:
$A^+ = \{A, B, C, D, E\}$
Therefore:
$A^+ = \{A, B, C, D, E\}$
So, A is a candidate key.
Step 2: Check for Partial Dependency
A relation violates 2NF when a non-prime attribute depends on only part of a composite candidate key.
Here, the candidate key is A, which is a single attribute.
Therefore, there cannot be a partial dependency.

- A → B,C → full dependency on the key A
- A → D → derived through A → BC and BC → D
- A → E → derived through D → E

Thus, no partial dependency exists.
Step 3: 2NF Decomposition
Since there are no partial dependencies, no decomposition is required.
is already in 2NF.
Final Answer

Item	Result
Candidate Key	A
Partial Dependency	None
2NF Violation	None
Decomposition	Not required
2NF Relation	R(A, B, C, D, E)

Important: The FD D → E is a transitive dependency, not a partial dependency. It becomes relevant when converting the relation from 2NF to 3NF, not when eliminating partial dependencies for 2NF

4. Apply 3NF decomposition on the relation EMPLOYEE(EmpID, EmpName, DeptID, DeptName, ManagerID) with transitive dependencies.

3NF Decomposition
Given relation:
EMPLOYEE(EmpID, EmpName, DeptID, DeptName, ManagerID)
Step 1: Identify Functional Dependencies
Assume the following dependencies:

1. **EmpID → EmpName, DeptID**
2. **DeptID → DeptName, ManagerID**

Therefore, we have a transitive dependency:
EmpID → DeptID → DeptName, ManagerID
Here, DeptName and ManagerID depend on EmpID **through DeptID**.
Step 2: Identify the Candidate Key
EmpID uniquely identifies an employee.
Therefore:
Candidate Key = EmpID
Step 3: Identify the Transitive Dependency
We have:
EmpID → DeptID
and
DeptID → DeptName, ManagerID
Hence:
EmpID → DeptName, ManagerID
is a **transitive dependency**.
This violates **3NF** because non-key attributes (DeptName, ManagerID) depend on another non-key attribute (DeptID).
Step 4: Decompose the Relation
Create a separate department relation:
DEPARTMENT(DeptID, DeptName, ManagerID)
Keep employee-specific information in:
EMPLOYEE(EmpID, EmpName, DeptID)
Final Decomposition
EMPLOYEE

EmpID	EmpName	DeptID
E101	Ravi	D01
E102	Priya	D02

DEPARTMENT

DeptID	DeptName	ManagerID
D01	HR	M01
D02	IT	M02

Keys

- **EMPLOYEE:** EmpID → Primary Key
- **DEPARTMENT:** DeptID → Primary Key
- EMPLOYEE.DeptID → Foreign Key referencing DEPARTMENT.DeptID

Final Answer
The transitive dependency **EmpID → DeptID → DeptName, ManagerID** is eliminated, so the resulting relations are in **3NF**.

5.Verify whether R(A,B,C,D) with FDs: AB→C, C→D, D→A is in BCNF. If not, decompose it into BCNF.

BCNF Verification and Decomposition
Given:
R(A, B, C, D)
Functional Dependencies:

1. AB → C
2. C → D
3. D → A

Step 1: Find Candidate Keys
Key 1: AB
From:
AB → C
and
C → D
Therefore:
AB^+ = {A, B, C, D}
So, AB is a candidate key.
Key 2: BC
From:
C → D
and:
D → A
So:
BC^+ = {B, C, D, A}
Therefore, BC is also a candidate key.
Key 3: BD
From:
D → A
and now we have A and B:
AB → C
Thus:
BD^+ = {B, D, A, C}
So, BD is also a candidate key.
Therefore, candidate keys are:
AB, BC, BD

Step 2: Check BCNF
A relation is in BCNF if, for every non-trivial FD X → Y, X must be a superkey.
Check each FD:

FD	Is determinant a superkey?	BCNF
AB → C	Yes, AB is a candidate key	✓
C → D	No	❌
D → A	No	❌

Therefore, R is NOT in BCNF.

Step 3: Decompose Using C → D
Since C → D violates BCNF, decompose R into:
R1(C, D)
and
R2(A, B, C)

R1(C, D)
FD:
C → D
Here, C is a candidate key of R1, so R1 is in BCNF.
R2(A, B, C)
The relevant FD is:
AB → C
Here, AB is a candidate key of R2, so R2 is in BCNF.
The dependency D → A is not preserved in these two relations, but the decomposition is lossless because the common attribute is C and:
C → D
determines all attributes of R1.
Final BCNF Decomposition
Therefore, the original relation is not in BCNF, and its BCNF decomposition is R1(C,D) and R2(A,B,C).

6. Experimentally verify lossless-join decomposition for a given relation using the tabular (Chase) algorithm

Experimental Verification of Lossless-Join Decomposition Using Chase Algorithm

Aim

To experimentally verify whether a decomposition of a relation is **lossless-join** using the **tabular (Chase) algorithm**.

Given Example

Let:

R(A, B, C)

Functional dependency:

A → B

Decompose R into:

- **R_1(A, B)**
- **R_2(A, C)**

We need to check whether this decomposition is **lossless**.

Step 1: Create the Chase Table

Create one row for each decomposed relation and one column for every attribute.

Use:

- **a_1, a_2, a_3** for distinguished symbols.
- **b_1, b_2, b_3** for non-distinguished symbols.

Row	A	B	C
R_1(A,B)	a_1	a_2	b_3
R_2(A,C)	a_1	b_2	a_3

For attributes present in a relation, use the corresponding **a-symbol**. For attributes absent from the relation, use a different **b-symbol**.

Step 2: Apply the Functional Dependency

Given:

A → B

Both rows have the same value in column **A**:

$a_1 = a_1$

Therefore, according to **A → B**, their B values must be made equal.

Initially:

- Row 1: B = **a_2**
- Row 2: B = **b_2**

Since A values are equal, replace **b_2** with **a_2**.

The table becomes:

Row	A	B	C
R_1(A,B)	a_1	**a_2**	b_3
R_2(A,C)	a_1	**a_2**	a_3

Step 3: Check for a Distinguished Row

After applying the FD, check whether **any row contains only distinguished symbols** (a_1, a_2, a_3).

Row 2 contains:

a_1, a_2, a_3

Therefore, a complete distinguished row exists.

Result

Since a row contains **all distinguished symbols**, the decomposition is **lossless-join**.

Conclusion

The **Chase algorithm experimentally verifies** that the decomposition is **lossless**, because the chase process produces a row containing all distinguished symbols.

7. Identify the multi-valued dependencies in TEACHER(TID, Subject, Hobby) and decompose it into 4NF.

4NF Decomposition of TEACHER
Given relation:
TEACHER(TID, Subject, Hobby)
Assume a teacher can teach multiple subjects and can have multiple hobbies, and these two sets are independent.

Step 1: Identify the Multi-Valued Dependencies

The multi-valued dependencies are:
TID →→ Subject
and
TID →→ Hobby
This means:

- A teacher can have multiple subjects.
- A teacher can have multiple hobbies.
- Subjects and hobbies are independent of each other.

Example
Suppose teacher T01 teaches DBMS and AI, and has hobbies Cricket and Music.
The original relation stores all combinations:

TID	Subject	Hobby
T01	DBMS	Cricket
T01	DBMS	Music
T01	AI	Cricket
T01	AI	Music

This creates unnecessary redundancy.

Step 2: Check 4NF

A relation is in 4NF if, for every non-trivial MVD:
X →→ Y
X must be a superkey.
Here:
TID →→ Subject
but TID is not a superkey of the original relation because it does not uniquely determine a single Subject-Hobby combination.
Therefore, TEACHER is not in 4NF.

Step 3: Decompose

Using the MVD:
TID →→ Subject
decompose the relation into:
TEACHER_SUBJECT(TID, Subject)
and
TEACHER_HOBBY(TID, Hobby)

Final Relations

TEACHER_SUBJECT

TID Subject

T01 DBMS

T01 AI

TEACHER_HOBBY

TID Hobby

T01 Cricket

T01 Music

Step 4: Verify 4NF

- In TEACHER_SUBJECT, TID →→ Subject is trivial because the relation contains only TID and Subject.
- In TEACHER_HOBBY, TID →→ Hobby is also trivial.

Therefore, both relations satisfy 4NF.

Final Answer

Conclusion: The original TEACHER relation violates 4NF due to the independent multi-valued dependencies TID →→ Subject and TID →→ Hobby. Decomposing it into the two relations above removes the redundancy and gives a 4NF decomposition.

8.Demonstrate how join dependencies lead to redundancy. Decompose a given relation to 5NF (Project-Join Normal Form).

5NF Decomposition Using Join Dependency

Example Relation

Consider:

SUPPLY(Supplier, Part, Project)

Suppose a supplier supplies a particular **Part** to a particular **Project**.

Assume the relationship is completely determined by three independent binary relationships:

- Supplier supplies Part
- Supplier works with Project
- Part is used in Project

Step 1: Identify the Redundancy

Suppose we have:

Supplier	Part	Project
S1	P1	J1
S1	P1	J2
S1	P2	J1
S1	P2	J2

Here, storing all three attributes together causes **redundancy**.

For example, if S1 supplies P1 and P2, and both parts are used in J1 and J2, several combinations have to be stored repeatedly.

This can be represented by the join dependency:

Step 2: Decompose the Relation

Break the relation into three smaller relations:

1. SUPPLIER_PART

SP(Supplier, Part)

Supplier	Part
S1	P1
S1	P2

2. SUPPLIER_PROJECT

SJ(Supplier, Project)

Supplier	Project
S1	J1
S1	J2

3. PART_PROJECT
PJ(Part, Project)

Part	Project
P1	J1
P1	J2
P2	J1
P2	J2

Step 3: Join the Relations
When we join:
SP ⋈ SJ ⋈ PJ
we get:

Supplier	Part	Project
S1	P1	J1
S1	P1	J2
S1	P2	J1
S1	P2	J2

Thus, the original information can be reconstructed without storing the redundant ternary combinations.
Step 4: Why This Is 5NF
A relation is in **5NF (Project-Join Normal Form)** when every non-trivial **join dependency** is implied by the candidate keys.
The original SUPPLY relation has a non-trivial join dependency, so it can be decomposed into the three relations above.
Final Decomposition
Conclusion
Join dependencies can cause redundancy when a relationship involving three or more attributes can be represented by smaller independent relationships. Decomposing SUPPLY into the three binary relations eliminates this redundancy and gives a **5NF decomposition**.

9.Given a poorly designed database schema for a college, identify all anomalies (insertion, deletion, update) and normalize it to 3NF.

Poorly Designed Relation

Consider:

COLLEGE(StudentID, StudentName, CourseID, CourseName, InstructorID, InstructorName, Grade)

Example:

StudentID	StudentName	CourseID	CourseName	InstructorID	InstructorName	Grade
S01	Ravi	C01	DBMS	I01	Kumar	A
S01	Ravi	C02	AI	I02	Priya	B
S02	Arun	C01	DBMS	I01	Kumar	A+

Functional Dependencies

1. StudentID → StudentName
2. CourseID → CourseName, InstructorID
3. InstructorID → InstructorName
4. (StudentID, CourseID) → Grade

The candidate key is:

(StudentID, CourseID)

A. Insertion Anomaly

Suppose a new course C03 = Computer Networks is introduced, but no student has enrolled yet.

We cannot store the course information without providing a StudentID.

Problem: Course information cannot be inserted independently.

B. Update Anomaly

Suppose instructor I01 Kumar changes their name/details.

The instructor's information may occur in many rows.

If we update only some rows, inconsistent information will exist.

Problem: The same information must be updated in multiple places.

C. Deletion Anomaly

Suppose student S01 is the only student enrolled in course C02.

If S01's enrollment is deleted, information about C02 and its instructor may also be lost.

Problem: Deleting enrollment can unintentionally delete course information.

Normalize to 3NF

Step 1: Remove Student Dependency

Create:

STUDENT(StudentID, StudentName)

StudentID	StudentName
S01	Ravi
S02	Arun

Step 2: Remove Course Dependency

Create:

COURSE(CourseID, CourseName, InstructorID)

CourseID	CourseName	InstructorID
C01	DBMS	I01
C02	AI	I02

Step 3: Remove Instructor Dependency

Create:

INSTRUCTOR(InstructorID, InstructorName)

InstructorID	InstructorName
I01	Kumar
I02	Priya

Step 4: Keep Student-Course Relationship

Create:

ENROLLMENT(StudentID, CourseID, Grade)

StudentID	CourseID	Grade
S01	C01	A
S01	C02	B
S02	C01	A+

Final 3NF Schema

Result: Partial and transitive dependencies are removed, so the database is in 3NF.

10. Compute the closure of attribute sets for the given FDs and determine all candidate keys for the relation.

For this question, a specific relation and FDs are required. Since they are not provided, consider the following standard example:
R(A, B, C, D, E)
Functional dependencies:

- **A → B**
- **B → C**
- **CD → E**
- **E → A**

We will find closures and candidate keys.
1. Find A^+
Start:
A^+ = {A}
Using **A → B**:
A^+ = {A, B}
Using **B → C**:
A^+ = {A, B, C}
No more dependencies can be applied.
Therefore:
A is **not** a candidate key because it does not determine D and E.

2. Find AD^+
Start:
AD^+ = {A,D}
Using A → B:
{A,B,D}
Using B → C:
{A,B,C,D}
Using CD → E:
{A,B,C,D,E}
Therefore:
So **AD is a superkey**.
Neither A nor D alone is a key, therefore:

3. Find BD^+
Start:
BD^+ = {B,D}
B → C:
{B,C,D}
CD → E:
{B,C,D,E}
E → A:
{A,B,C,D,E}
Therefore:
So:

4. Find CD^+
Start:
CD^+ = {C,D}
Using CD → E:
{C,D,E}

Using E → A:
{A,C,D,E}
Using A → B:
{A,B,C,D,E}
Therefore:
So:

Final Candidate Keys
The candidate keys are:
Simple Method to Remember
Attribute Closure:
Start with the given attributes → repeatedly apply applicable FDs → continue until no new attributes can be added.
If the closure contains **all attributes of R**, the attribute set is a **superkey**. If no attribute can be removed while retaining that property, it is a **candidate key**.